

Software Architecture
(T630019402)

Lecture #6, March 17, 2026

Jukka Ruohonen

Case #5 Presentations

- ▶ Although no longer maintained, Hystrix was at one point a fault-tolerance framework used by Netflix
- ▶ Study the framework and its documentation (<https://github.com/Netflix/Hystrix>)
 - What were their **scenarios**?
 - What **tactics** they used?
 - How about **patterns**?

Today's Lecture

- ▶ Today's lecture is about **performance** as a distinct QA

Question

How would you characterize performance?

Performance and Scalability

- ▶ Performance is about a *system's ability to meet resource requirements*; by following algorithmic complexity, we can consider “time performance” (timing requirements) and “space performance” (memory and disk use)
- ▶ Performance is related to **scalability**, which is about proportionally increasing performance by adding resources
- ▶ When considering performance as a standalone QA, we are interested in doing more with the resources we have, that is, without adding new resources

Performance Scenarios

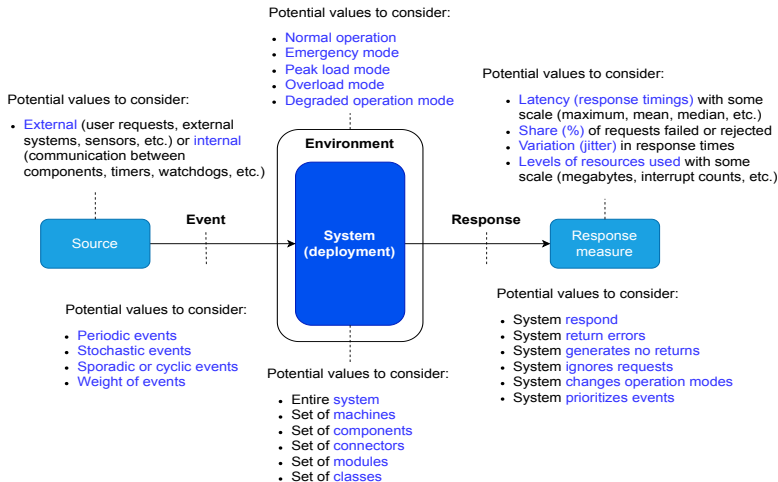


Figure 1: Adopted and modified from Bass et al. (2021)

Latency and Throughput

- ▶ Particularly when considering networking, including with respect to distributed systems, the two basic measures are:
 1. **Latency**: the time to perform an action or to produce a result
 2. **Throughput**: number of actions or results per unit of time
- ▶ The optimum is **maximal throughput with minimal latency**

Units of Time for Understanding Latency

- ▶ A **microsecond** is 1000 **nanoseconds** and 0.000001 **seconds**
- ▶ A rough order of things:
 1. L1 cache reference (less than a nanosecond)
 2. L2 cache reference (...)
 3. Mutex lock/unlock (...)
 4. L3 cache reference (...)
 5. Main memory reference (...)
 6. Send packet through a Unix domain socket (...)
 7. Read or write to a SSD disk (...)
 8. Send a TCP/IP packet within the same data center (...)
 9. Send a TCP/IP packet from Denmark to Japan (...)
 10. Watch Netflix (...)

Context Matters for Understanding Latency

- ▶ From the previous listing, it can be understood that latency varies a lot between different contexts
- ▶ While operating systems kernels operate with nanoseconds, networked connections *within the same data center* are generally much faster than general Internet connections
- ▶ However, networks vary a lot; from cellular networks and terrestrial optical fiber network to satellite communications
 - Handley's (2018) early results indicated that satellite communications may have **latency spikes** even up to 100 seconds; hence, it should be also stressed that latency is **often stochastic rather than deterministic** in its nature

The Eight Fallacies of Distributed Computing

- ▶ Drawing from Deutsch's list from the early 1990s, Wilson (2015) listed the following fallacies of distributed computing:
 1. Networks are reliable
 2. Latency is zero
 3. Bandwidth is infinite
 4. Networks are secure
 5. Topology does not change
 6. There is only one administrator
 7. Transport cost is zero
 8. Network is homogenous

Performance Tactics

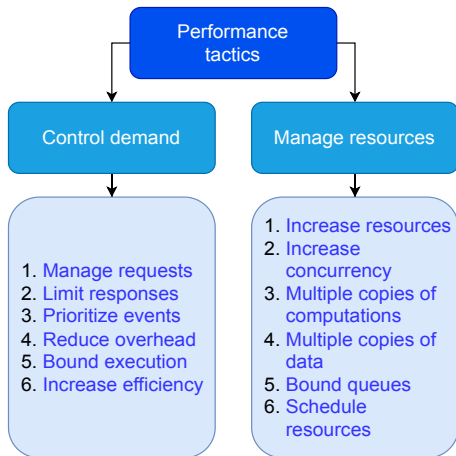


Figure 2: Performance Tactics

Performance Tactics: Demand $\mapsto$ Manage requests (1/3)

- ▶ There are two sides to the tactics: **demand and supply**
- ▶ On the demand-side, one is basically trying to limit the amount of requests coming to a system, possibly without even alternating the system in any way
- ▶ Many – if not most – Internet-based systems assume and define some **maximum of requests** that a given system is able or willing to support
 - These are often defined with **terms of service** (ToS) or a specific **service-level agreement** particularly in the context of business-to-business (B2B) services

Performance Tactics: Demand $\mapsto$ Manage requests (2/3)

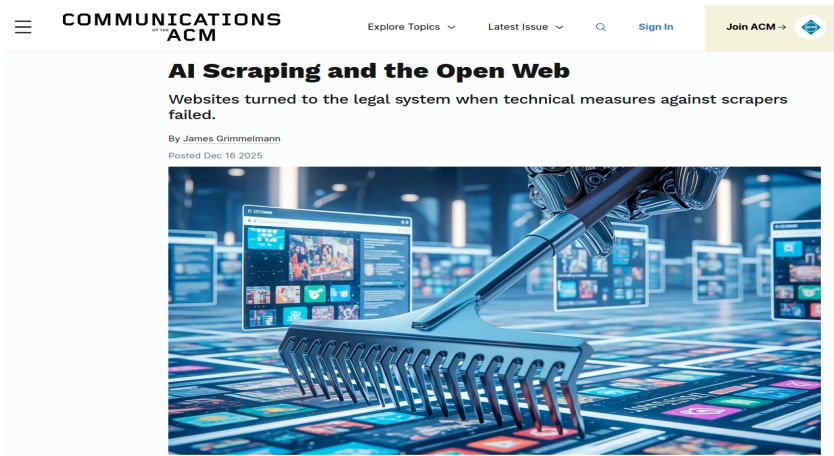


Figure 3: <https://cacm.acm.org/opinion/ai-scraping-and-the-open-web/>

Performance Tactics: Demand $\mapsto$ Manage requests (3/3)

- ▶ In some contexts, it is also possible to define and limit the **sampling frequency** of the arriving requests
- ▶ For instance, a video steaming service might reduce its fidelity or sensors could be defined to send requests only periodically instead of continuously
 - Of course, this solution involves an overall quality-of-service (QoS) **trade-off** between latency and quality

Performance Tactics: Demand $\mapsto$ Limit responses (1/2)

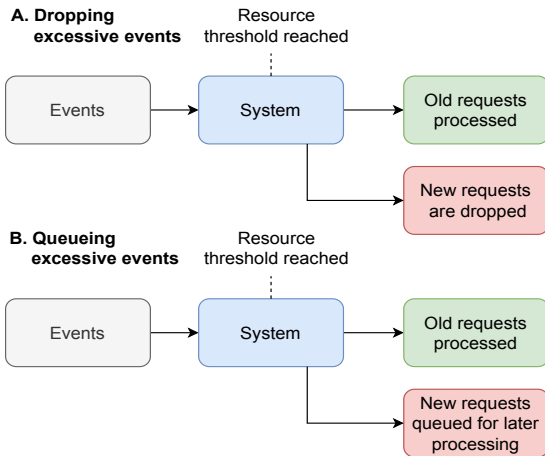


Figure 4: Two Basic Solutions to Limit Responses

Performance Tactics: Demand $\mapsto$ Limit responses (2/2)

- ▶ The **out-of-memory** (OOM) killer in Linux is a good example about limiting “responses” out of necessity: if there is not enough memory, including after disk-based swap has been exhausted, the Linux kernel starts to kill processes, generally picking the most memory-hungry processes first
- ▶ But sometimes it may be desirable to crash loudly upon OOM:

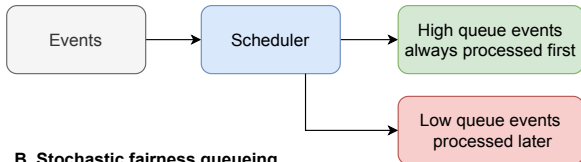
```
$ cat /proc/sys/vm/panic_on_oom  
1
```


Performance Tactics: Demand $\mapsto$ Prioritize events (1/2)

- ▶ A common tactic involves assigning a priority value to events
 - FIFOs and LIFOs from the previous lecture are examples
- ▶ As a more involved example: the network stacks of many operating systems can be configured to use various different queues for incoming packets, including **priority queues** and **stochastic fairness queueing** (Cho 1999)
- ▶ There are trade-offs, including with respect to **fairness** (which events should be prioritized?) and **overhead** (if you are sure that a resource threshold will never be exceeded, event prioritization actually decreases performance)

Performance Tactics: Demand $\mapsto$ Prioritize events (2/2)

A. Priority queueing



B. Stochastic fairness queueing

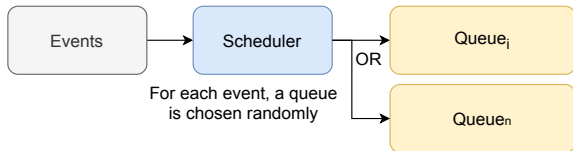
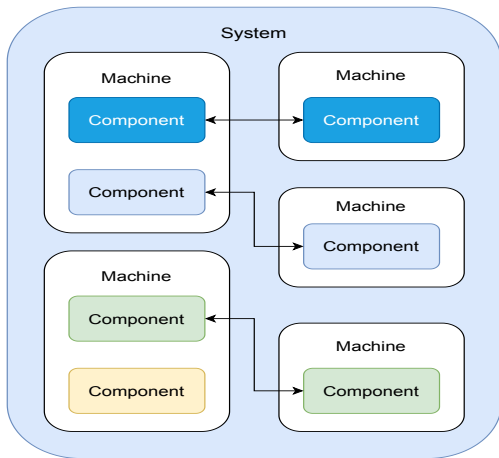


Figure 5: A High-Level Illustration of Two Queueing Tactics

Performance Tactics: Demand $\mapsto$ Reduce overhead (1/3)

- ▶ Overhead can be often simply reduced by **decreasing the amount of intermediaries** used
 - However, there is a **trade-off with maintainability**
- ▶ Maybe *numerous wrappers* were not a good idea after all? Maybe you had *too many filters* in your pipe-and-filter pattern? Perhaps you could *reduce the number of stages* in your batch-sequential pattern? Or could it be that you chose a *new shiny thing* for your system without considering its performance impacts?

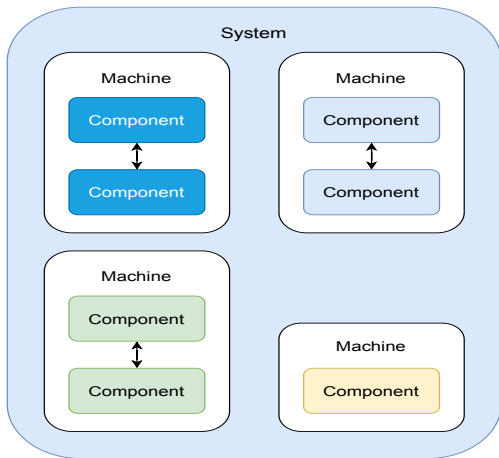
Performance Tactics: Demand $\mapsto$ Reduce overhead (2/3)



Network communication increases **latency**, but the same co-location ideas apply also more generally.

Figure 6: An Illustration of Overhead Due to Networking

Performance Tactics: Demand $\mapsto$ Reduce overhead (3/3)



Co-locating resources applies to other many contexts as well.

Figure 7: Reduced Latency Due to Co-locating Components

Performance Tactics: Demand $\mapsto$ Bound Execution

- ▶ Setting **caps** to execution resources, whether time, memory, or something else, is a good idea for reserving **spare capacity**
- ▶ A machine learning algorithm might not need tens of thousands of iterations to complete, for instance
- ▶ Resource limits are a good idea also for **security** reasons

setrlimit(2) - NetBSD Manual Pages

COMMAND: SECTION: ARCH: COLLECTION:

GETRLIMIT(2) NetBSD System Calls Manual GETRLIMIT(2)

NAME

getrlimit, setrlimit -- control maximum system resource consumption

Figure 8: <https://man.netbsd.org/setrlimit.2>

Performance Tactics: Demand $\mapsto$ Increase Efficiency

- ▶ Last but not least, performance can be obviously obtained by using **better algorithms** and generally **optimizing** programs
- ▶ Yet, there is often again a **trade-off with maintainability**
 - For instance, sometimes routines in kernels have been written in assembly, but later these have been deprecated because a few other people have understood the routines
- ▶ A similar trade-off applies to **portability**
 - If you write x86 assembly to optimize a video game engine, what happens if you need to later port the engine to ARM?

Performance Tactics: Resources $\mapsto$ Add resources

- ▶ Although adding resources was earlier framed outside to “the pure performance QA”, obviously also performance can be improved with better hardware, faster networking equipment, sometimes GPUs instead of CPUs, and so forth and so on
- ▶ While adding resources is perhaps more about **scalability**, it can be noted that often (but not always) new resources cost less than large-scale restructuring and optimization of a large and complex system

Performance Tactics: Resources $\mapsto$ Concurrency (1/7)

- ▶ Although dictionary definitions for the adjective *concurrent* emphasize both things occurring at the same time (simultaneous occurrence) and in parallel, we typically separate **concurrency** from **parallelism**¹
- ▶ The reason is that parallelism can be seen to require multiple CPUs, whereas concurrency can be done with a single CPU
- ▶ In conventional programming with a single machine, it is sensible to further simplify things by associating concurrency with **threads** and parallelism with **processes**
 - But note that the distinction is not accurate at the operating system level due the emergence of multi-core CPUs

¹ <https://www.merriam-webster.com/dictionary/concurrent>

Performance Tactics: Resources $\mapsto$ Concurrency (2/7)

A. The terminological simplification

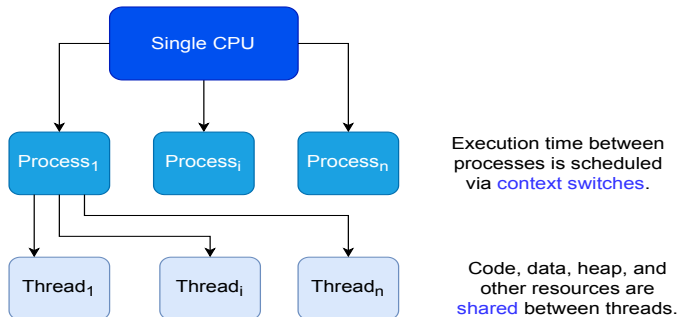


Figure 9: The Terminological Simplification

Performance Tactics: Resources $\mapsto$ Concurrency (3/7)

B. In reality, modern machines run both concurrently and in parallel

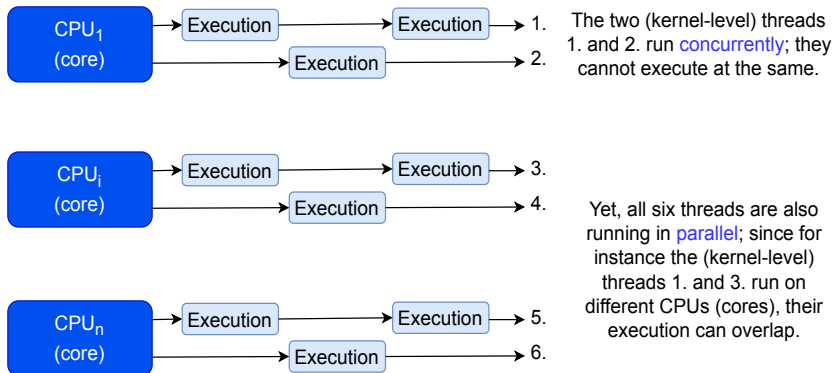
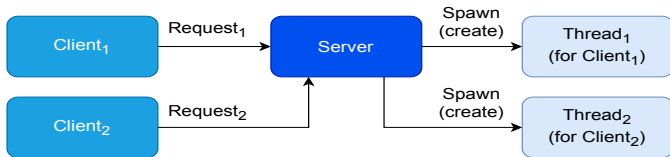


Figure 10: The Terminological Simplification Revisited

Performance Tactics: Resources $\mapsto$ Concurrency (4/7)

A. Spawning a new thread for each request



B. Using a thread pool

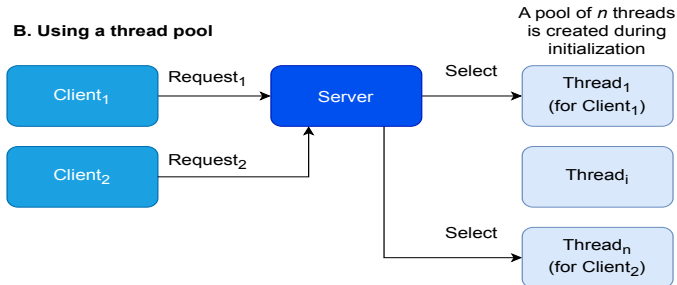


Figure 11: Two Basic Strategies for Creating Threads

Performance Tactics: Resources $\mapsto$ Concurrency (5/7)

```
import queue as queue
import threading as threading

class Parent: # Thread pool

    def __init__(self):

        self.__maxThreads = 500
        self.queue = queue.Queue()    # Asynchronous
        self.lock = threading.Lock() # "Mutex"

        for i in range(self.__maxThreads):
            t = self.__Worker(self)
            t.setDaemon(True)
            t.start()
```

Performance Tactics: Resources $\mapsto$ Concurrency (6/7)

```
class __Worker(threading.Thread): # Sub-class.

    def __init__(self, parent):
        threading.Thread.__init__(self)
        self.parent = parent

    def run(self):

        while True:
            buf = self.parent.queue.get()
            self.parent.lock.acquire()
            self.__store(buf) # Shared storage
            self.parent.lock.release()
            self.parent.queue.task.done()
```

Performance Tactics: Resources $\mapsto$ Concurrency (7/7)

- ▶ In reality, however, the first design choice is about carefully contemplating whether threads are even the best solution
- ▶ For instance, the classical Apache web server uses a thread pool, whereas alternatives, including Lighttpd and Nginx, use a single thread with an **asynchronous event-driven** approach
 - Historically, the asynchronous approaches for web servers became via the early 2000s operating system innovations²
- ▶ Nevertheless, there are numerous scenarios with which even spawning new threads on demand is a sensible choice

² <https://man.freebsd.org/cgi/man.cgi?kqueue>

Performance Tactics: Resources $\mapsto$ Copies of computations

- ▶ **Redundancy** can be perceived as being *also* about maintaining computation results
 - The read-write replicas from the previous lecture are a good example also about a performance tactic: it is often a good idea to separate **infrequent writes** into a distinct component (including a database), replicate these to read-only components (including databases), and then serve the **frequent reads** from the replicated read-only components
 - For instance, a machine learning model's results might be read often for making predictions, but written only seldom as retraining can take a lot of resources
- ▶ **Load balancers** and **server pools** are further examples

A Recap from the Previous Lecture

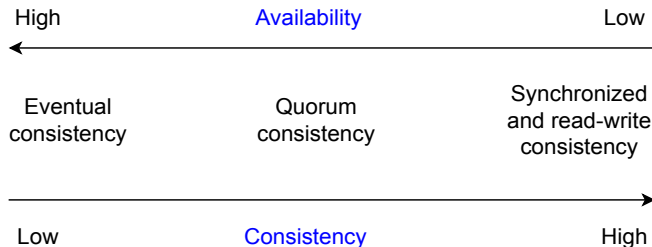


Figure 12: A Summary of the Consistency Types

Performance Tactics: Resources $\mapsto$ Copies of data (1/8)

- ▶ For maintaining data redundancy, one must start from the fallacy that networks are reliable, when, in fact, they are not
- ▶ Thus, a brief revisit to **availability** themes before continuing
- ▶ The **CAP theorem** (Brewer 2000) states that any distributed system can only provide two of the following three guarantees:
 1. **Consistency**: every read always receives either the most recent write or an error
 2. **Availability**: every request receives a response but without guarantees that the response contains the most recent data
 3. **Partition tolerance**: a system continues to operate despite arbitrary partitioning due to network failures

Performance Tactics: Resources $\mapsto$ Copies of data (2/8)

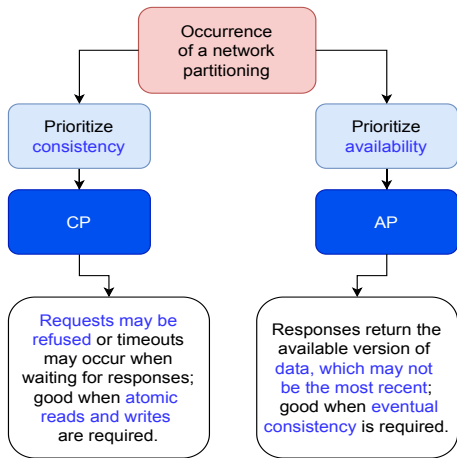


Figure 13: The Underlying Trade-Off in the CAP Theorem

Question

Would you prefer CP or AP,
and how would you solve the trade-off?

Performance Tactics: Resources $\mapsto$ Copies of data (3/8)

- ▶ A classical way to approach the CAP trade-off is **segregation**
- ▶ For instance, in the previous lecture it was suggested that availability should be guaranteed for **payment systems**
 - However, perhaps you would rather agree that prioritizing atomicity and consistency would be the optimal tactic; after all, you do not want inconsistencies in payment transactions?
- ▶ From a segregation viewpoint, there are distinct components that require **prioritizing either availability or consistency**
 - For instance, on Instagram people likely want to see people's pictures regardless whether they are the most recent
 $\implies$ Prioritize availability (AP)

Performance Tactics: Resources $\mapsto$ Copies of data (4/8)

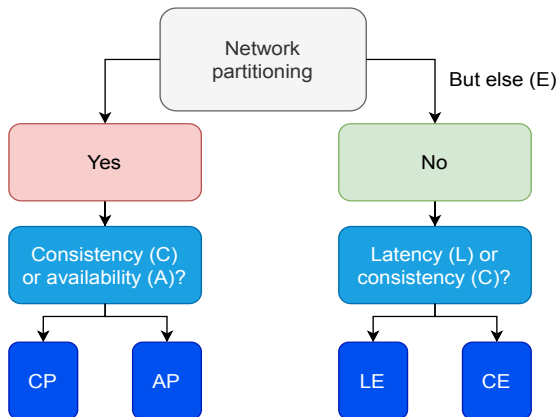


Figure 14: The Trade-Offs in the PACELC Theorem (Abadi 2010)

Performance Tactics: Resources $\mapsto$ Copies of data (5/8)

- ▶ One way to frame **caching** is to consider where a cache is **located**; yet, when considering distributed systems, it is sensible to frame distributed caches as those that are located at different machines, thus requiring a network

Table 1: Local and Distributed Caches

Characteristic	Local caches	Distributed caches
Location	Same machine	Different machines
Network	Not required	Required
Sharing	Not shared	Shared
Access speed	Fast	Slow (latency)
Scalability	Limited	Scalable

Performance Tactics: Resources $\mapsto$ Copies of data (6/8)

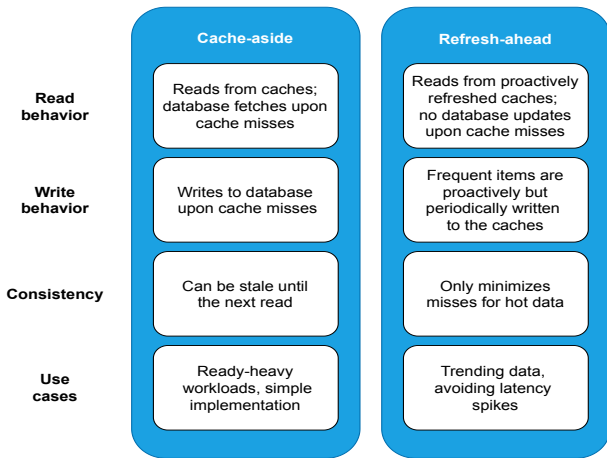


Figure 15: Two Types of Caches (adopted from Slyus 2025)

Performance Tactics: Resources $\mapsto$ Copies of data (7/8)

	Write-through	Write-back	Write-around
Read behavior	Reads from cache, always up-to-date	Reads from cache, may have a small lag	Reads from database initially, then from cache
Write behavior	Writes to cache and database simultaneously	Writes to cache, database only updated later	Writes directly to database, bypasses cache
Consistency	Strong consistency, always up-to-date	Risk of data loss in case cache crashes	First read slower, then consistent
Use cases	Strong consistency required, critical data	Write-heavy workloads, performance maximized	Infrequent reads, minimizes memory consumption

Figure 16: Three Further Cache Types (adopted from Slyus 2025)

Performance Tactics: Resources $\mapsto$ Copies of data (8/8)

	Write-through	Write-back	Write-around
Write latency	High	Low	Medium
Consistency	Strong	Eventual	Strong
Risk of data loss	None	Possible	None
Examples	Financial transactions	Analytics, social media	Write-heavy cases with rare reads

Figure 17: Trade-Offs With the Previous Three Cache Types

Performance Tactics: Resources $\mapsto$ Bound queues (1/2)

- ▶ The bound queues tactic limits the **maximum number of queued events**; while simple to implement, a decision is again needed about what to do when overflows occur
 - One solution is to reject events stochastically once a queue's length exceeds a threshold (cf. Cho 1999, p. 2)

```
def enqueue(self, events, max = 1000):  
  
    k = max(len(events), max)  
  
    for i in range(k):  
        self.queue.put_nowait(events[i])
```

Performance Tactics: Resources $\mapsto$ Bound queues (2/2)

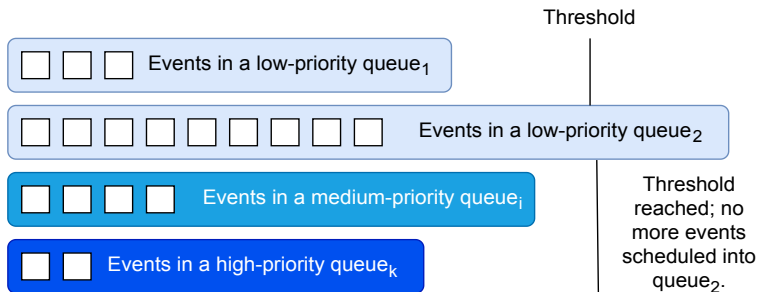


Figure 18: Illustrating Thresholds (Circuit Breakers) for Queues

Performance Tactics: Resources $\mapsto$ Schedule (1/12)

- ▶ **Scheduling resources** is a fundamental but challenging technique to increase performance via managing resources
- ▶ When scheduling, we generally **do not want to hop between tasks** because *context switches are usually expensive*
 - That is the reason why the previous example was not only about thresholds and circuit breakers but also about **priorities**

Performance Tactics: Resources $\mapsto$ Schedule (2/12)

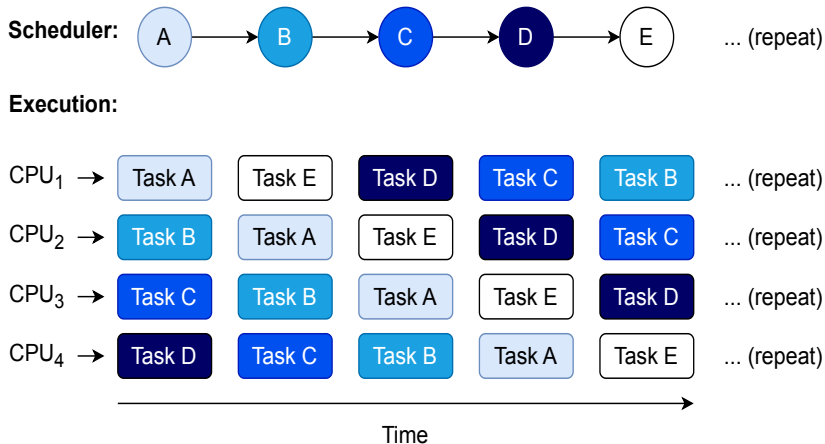


Figure 19: A Poor CPU Scheduler (from Arpaci-Dusseau et al. 2023)

Performance Tactics: Resources $\mapsto$ Schedule (3/12)

Execution:

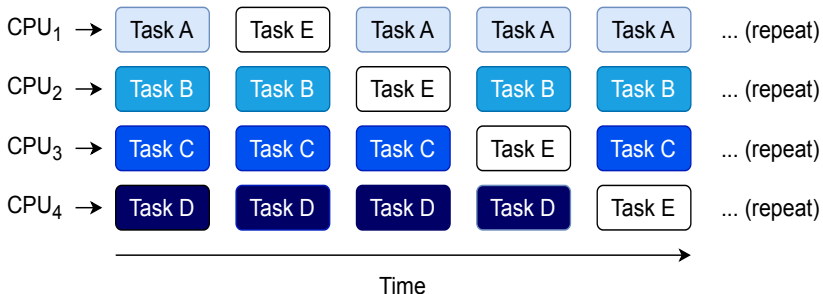
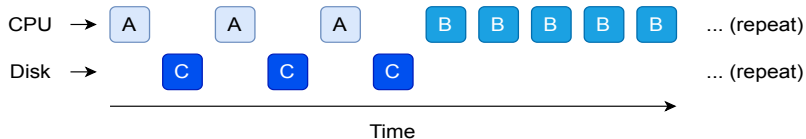


Figure 20: A Better CPU Scheduler (from Arpaci-Dusseau et al. 2023)

Performance Tactics: Resources $\mapsto$ Schedule (4/12)

Poor use of resources:



Better use of resources:

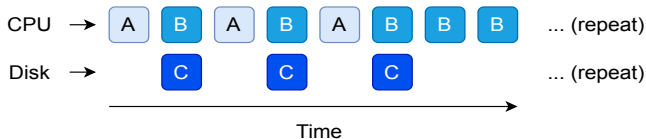


Figure 21: Overlapping Execution Is Good (Arpaci-Dusseau et al. 2023)

Performance Tactics: Resources $\mapsto$ Schedule (5/12)

- ▶ The earlier queue examples already noted the **fairness** principle, and it is the *de facto* way most operating system kernels (including Linux) allocate CPU time to processes
- ▶ However, sometimes a fair distribution of resources is a poor choice; **real-time systems** are a prime example in this regard
 - In general, they require that an event *must be* completed in some time; say, the breaks of a car must work and must work almost immediately ($\implies$ the notion of “hard real-time”)
 - FIFOs and LIFOs are not usually good for real-time systems

Performance Tactics: Resources $\mapsto$ Schedule (6/12)

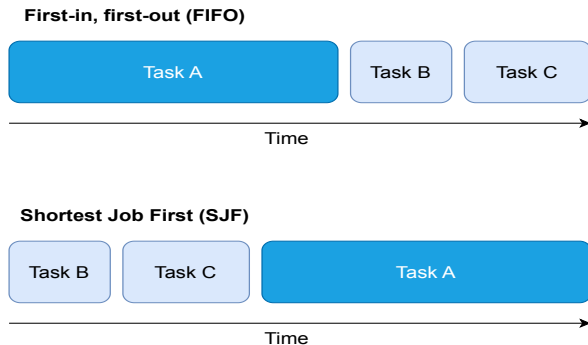


Figure 22: Why FIFO May or **May Not** Be a Good Idea

Performance Tactics: Resources $\mapsto$ Schedule (7/12)

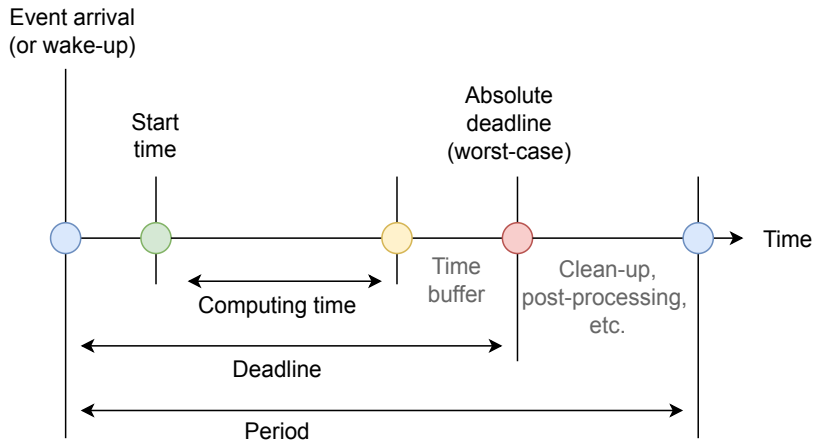


Figure 23: A Real-Time Scheduler (mostly as in the Linux kernel)

Performance Tactics: Resources $\mapsto$ Schedule (8/12)

- ▶ From the previous example, we can deduce that it is possible to prioritize tasks by deadlines; a **deadline monotonic** scheduler assigns higher (lower) priorities to processes (tasks) with shorter (longer) deadlines
 - The example also illustrates that the overall periods can be alternatively used for prioritization in which case a concept of **rate monotonic** scheduler is sometimes used
- ▶ In contrast, a scheduler based on **semantic importance** assigns priorities based on some domain characteristics
 - For instance, audio could be prioritized over video

Performance Tactics: Resources $\mapsto$ Schedule (9/12)

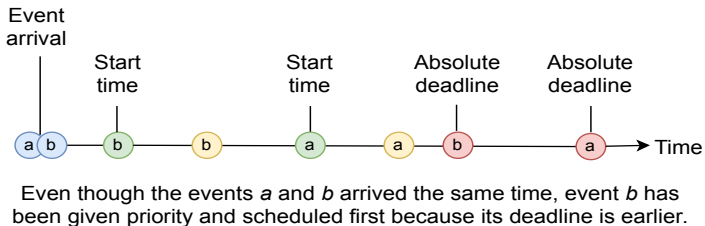


Figure 24: A Deadline Monotonic ("Earliest-Deadline-First") Scheduler

Performance Tactics: Resources $\mapsto$ Schedule (10/12)

- ▶ A further example is **idle scheduler**: it can be used to run some tasks that are not time-critical only when a system is idle (i.e., it has nothing else to do)
- ▶ Then, a **batch scheduler** can be used to maximize throughput by running batch jobs (e.g., backups) in the background with a low priority
- ▶ A **round-robin scheduler** assigns time slices for tasks in equal proportions and in a circular manner
 - It is easy to implement and commonly used; e.g., in DNS

Performance Tactics: Resources $\mapsto$ Schedule (11/12)

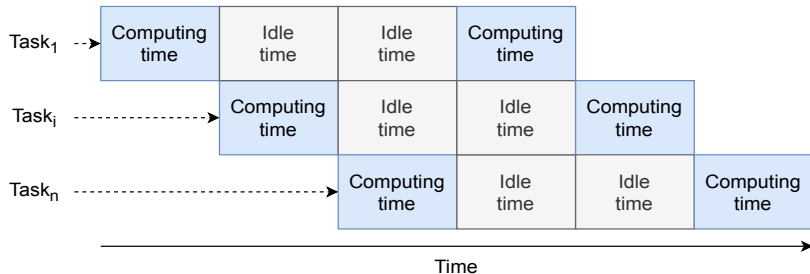


Figure 25: A Simple Round-Robin Scheduler

Performance Tactics: Resources $\mapsto$ Schedule (12/12)

- ▶ Finally, a given context defines the **granularity** of scheduling, including the appropriate time unit for latency measurements
 - While writes to a database might be measured in seconds (or more), a kernel of an operating system continuously schedules (in nanoseconds) not only processes but also hardware interrupts that need to be allocated to CPUs



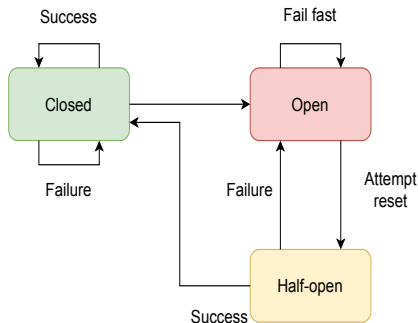
irqbalance(1) - Linux man page

Name

irqbalance - distribute hardware interrupts across processors on a multiprocessor system

Figure 26: <https://linux.die.net/man/1/irqbalance>

Another Recap from the Previous Lecture (1/2)



Closed: requests are passed to a service and faults increase counters. If a counter exceeded a threshold, the circuit breaker is tripped and transitions into the open state.

Open: requests are not passed to a service. Instead, all faults are passed to clients immediately. If responsiveness is again gained or ping-echos indicate so, the circuit breaker transitions into the half-open state.

Half-open: only a limited number of requests are passed to a service. If a service responds with successes, the circuit breaker transitions again into the closed state. But if requests still fail, the circuit breaker transitions back to the open state.

Figure 27: A State Diagram for the Circuit Breaker Pattern (adopted from Montesi & Weber 2016, Fig. 1 and p. 2)

Another Recap from the Previous Lecture (2/2)

The example circuit breaker monitors n similar resources with thresholds a and b .

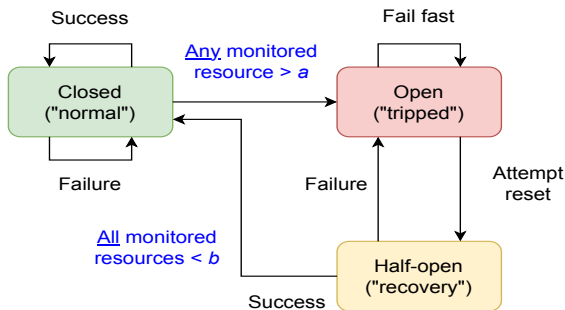


Figure 28: An Example of a Simple Circuit Breaker Design

Performance Patterns: Throttling (1/11)

- ▶ Many patterns, such as **service meshes** and **load balancers**, are on the side of scalability (at least with the framing done)
 - Thus, these will be covered in the next lecture
- ▶ For today, we will first briefly reconsider **throttling**, which has already been noted earlier but which is difficult to deduce and engineering without a context; throttling *power consumption* is rather different than throttling *network traffic*
 - With networks, metrics about requests (e.g., 1000 per second) and data (e.g., 2 GiB per minute) can be used
 - To motivate the next lecture and illustrate the alternatives to throttling, in what follows the context is **cloud computing**

Performance Patterns: Throttling (2/11)

- ▶ In terms of architectural patterns, throttling is usually implemented with the **circuit breaker** pattern; thus
 1. The **closed state** is the “normal” state; a circuit breaker is measuring load against a predefined threshold
 2. Throttling occurs in the circuit breaker’s **half-open state**: load is limited by graceful degradation
 3. The **open state** is for emergencies: throttling or other things did not help; the system will return errors to all clients
- ▶ For simplicity, we assume that the **threshold is static**, although some algorithms may involve also dynamic thresholds

Performance Patterns: Throttling (3/11)

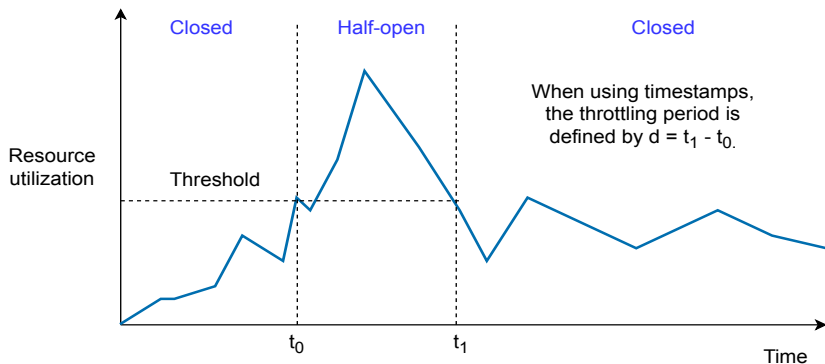


Figure 29: Throttling With a Circuit Breaker (cf. Microsoft 2026)

Performance Patterns: Throttling (4/11)

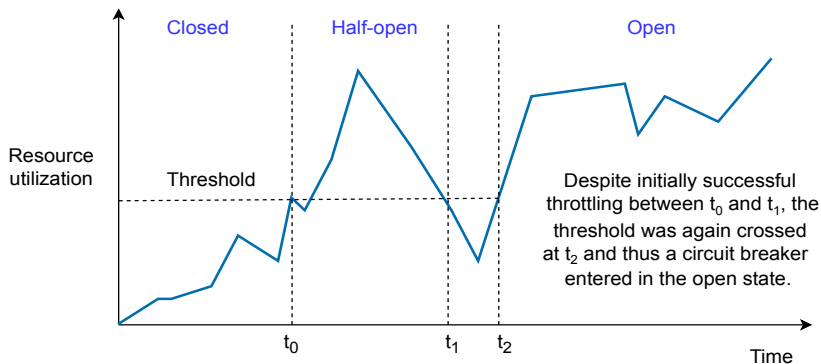


Figure 30: Throttling Did Not Help: **What to Do?**

Performance Patterns: Throttling (5/11)

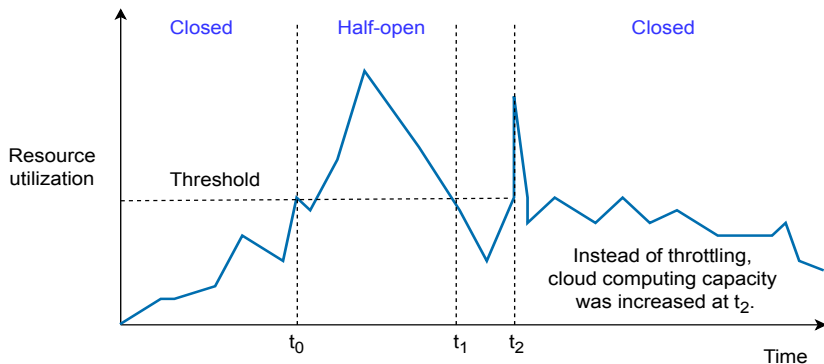


Figure 31: Combining Throttling With Dynamic Resource Allocation

Performance Patterns: Throttling (6/11)

- ▶ Thus, we may dynamically **scale up and down** in contexts such as cloud computing, but there are again trade-offs
 - When scaling down, we are reducing the QoS for our clients and in the worst case scenario cutting off availability for them
 - When scaling up, we are consuming more resources, and resource consumption costs money and time
- ▶ The dual dynamic scaling makes algorithmic solutions complex
 - Even without scaling up, simple circuit breakers have known problems with **retries**; once some clients are served errors or they timeout, they will typically retry their queries, *making the situation even worse*

Performance Patterns: Throttling (7/11)

- ▶ **Layered architectures** also contribute to the problem because there are typically multiple queries and responses between the internal layers; that is, the issue is not only between clients and the external-facing components
- ▶ Therefore, throttling is often implemented with an **exponential back-off** algorithm; every retry attempt increases waiting time for a client exponentially
- ▶ Another solution is to add some **jitter** (that is, randomness) to retry backoff times (and perhaps also to normal requests too)
- ▶ Brooker (2019) provides a good discussion on these topics

Performance Patterns: Throttling (8/11)

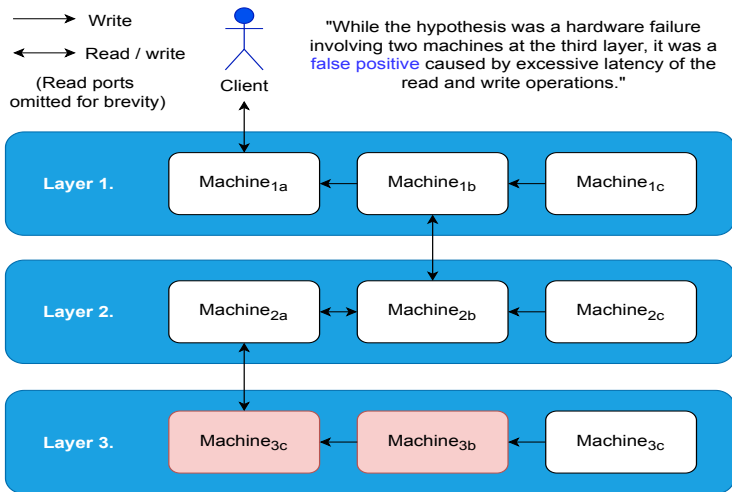


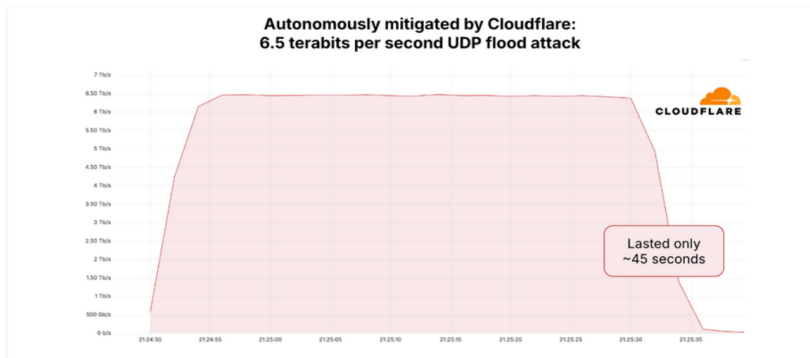
Figure 32: Latency May Increase Also the Probability of False Positives

Performance Patterns: Throttling (9/11)



Figure 33: <https://krebsonsecurity.com/2025/05/krebsonsecurity-hit-with-near-record-6-3-tbps-ddos/>

Performance Patterns: Throttling (10/11)



A graph depicting the 6.5 Tbps attack mitigated by Cloudflare in April 2025. Image: Cloudflare.

Figure 34: <https://krebsonsecurity.com/2025/05/krebsonsecurity-hit-with-near-record-6-3-tbps-ddos/>

Performance Patterns: Throttling (11/11)



Figure 35: <https://labs.ripe.net/author/mkoch/forward-to-hell-on-misusing-transparent-dns-forwarders-for-amplification-attacks>

Performance Patterns: MapReduce as an Example (1/2)

- ▶ Finally, it is worthwhile to return to the earlier remark about adding resources being often (but not always!) cheaper than optimization and algorithms
 - As always, use **healthy skepticism** with rules of thumbs (hence, note that the CAP theorem too is debated)
- ▶ Therefore, Google's MapReduce is a good final example on how algorithms are still used even in large data centers and large-scale computing clusters
 - The idea is simple: use standard programming concepts (maps, keys, and values) to reduce big data into manageable datasets (see Dean & Ghemawat 2008 for details)

Performance Patterns: MapReduce as an Example (2/2)

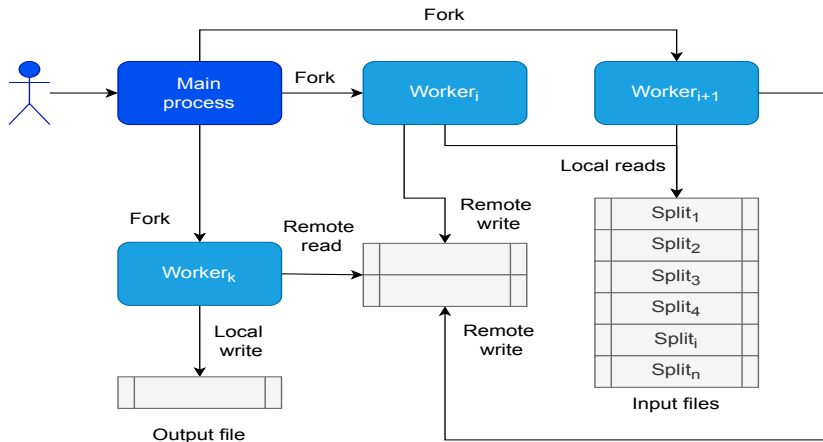


Figure 36: The Basic Idea of MapReduce (adopted, interpreted, and modified from Dean & Chemawat 2008, Fig. 1)

Performance Patterns: Claim-Check Pattern

► MapReduce aligns with a more general claim-check pattern

- With sound cryptography for the claim-check token functionality, **security** can improve.
- Since networks are unreliable, **reliability** increases due to the use of a local storage.
- **Performance** increases because large payloads are not transmitted via slow networks.

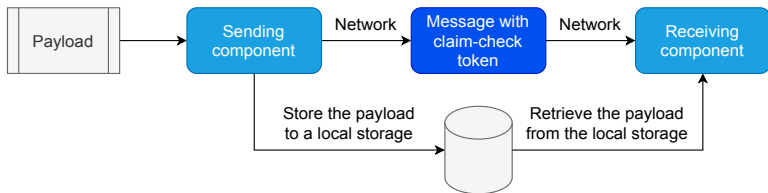


Figure 37: The Claim-Check Pattern (adopted from Microsoft 2025)

Performance Patterns: Data and Content Separation

- ▶ In web development a classical performance pattern involves separating **static content from dynamic content**, and then serving these from different servers and databases
 - Content delivery networks (CDNs) can be a good idea too; more about these in the next lecture
- ▶ In addition, Microsoft (2025) mentions a **materialized view pattern**; it is essentially about **pre-computing** “views” (or more generally results) for clients instead of always making heavy database queries or heavy computations
 - For instance, it may be a good idea to pre-compute complex SQL queries and put the results into separate tables

Performance Patterns: Databases

- ▶ Databases are a commonplace bottleneck
- ▶ In addition to the already noted **caching** and **read replicas**, considering **sharding** of large databases (e.g., separate European users into their own database)
- ▶ Learn how database **indexing** works
- ▶ Learn and debug database **locking** issues
- ▶ Use **simultaneous writes** for large batches
- ▶ Database connections are costly; thus, use **connection pools**
- ▶ And, finally, **optimize your queries** if necessary

Questions?

Case #6

- ▶ Write a performance scenario for a hypothetical *online auction platform serving the whole consumer market in India*
 - What would the **main concerns** be?
 - What would the **worst-case latency** and **worst-case throughput** cases be, and how would you *deal with them*?
 - What *tactics and patterns* would you use for **average-case latency and throughput** settings?
 - What the **CAP theorem** says about the scenario?

Thank you

Questions?

Additional References (1/2)

1. Handley, M. (2018): Delay is Not an Option: Low Latency Routing in Space. HotNets, pp. 85, 91,
<https://dl.acm.org/doi/pdf/10.1145/3286062.3286075>
2. Wilson, G. (2015): Eight Fallacies of Distributed Computing – Tech Talk.
<https://web.archive.org/web/20171107014323/http://blog.fogcreek.com/eight-fallacies-of-distributed-computing-tech-talk/>
3. Cho, K. (1999): Managing Traffic with ALTQ. FREENIX Track, USENIX Annual Technical Track, pp. 1–9, https://www.usenix.org/legacy/event/usenix99/full_papers/cho/cho.pdf
4. Brewer, E. A. (2000): Toward Robust Distributed Systems. Keynote at PODC, https://sites.cs.ucsb.edu/~rich/class/cs293b-cloud/papers/Brewer_podc_keynote_2000.pdf
5. Abadi, D. (2010): Problems With CAP, and Yahoo's Little Known NoSQL System. <https://dbmsmusings.blogspot.com/2010/04/problems-with-cap-and-yahoos-little.html>

Additional References (2/2)

1. Slys, J. (2000): Understanding Caching: A Beginner's Guide.
<https://iam.slys.dev/p/understanding-caching-a-beginners>
2. Microsoft (2026): Throttling Pattern. <https://learn.microsoft.com/en-us/azure/architecture/patterns/throttling>
3. Arpaci-Dusseau, R. H. & Arpaci-Dusseau, A. C. & Reiher, P. (2023): Operating Systems: Three Easy Pieces. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
4. Brooker, M. (2019): Timeouts, Retries, and Backoff with Jitter. Amazon Web Services, <https://d1.awsstatic.com/builderslibrary/pdfs/timeouts-retries-and-backoff-with-jitter.pdf>
5. Dean, J. & Ghemawat, S. (2008): MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM 51(1), pp. 107–113,
<https://dl.acm.org/doi/pdf/10.1145/1327452.1327492>
6. Microsoft (2025): Cloud Design Patterns.
<https://learn.microsoft.com/en-us/azure/architecture/patterns/>